

Codexion

El proyecto presenta una simulación de concurrencia donde varios *coders* trabajan en un espacio compartido. Cada uno alterna entre tres estados: *compiling*, *debugging* y *refactoring*. Para *compile*, cada coder necesita dos *USB dongles*, uno para cada mano. Los dongles son recursos limitados y deben compartirse sin provocar *deadlocks*, *starvation* o *burnout*.

La introducción explica cómo la programación moderna combina creatividad, colaboración y gestión de recursos, y cómo este proyecto modela esos desafíos mediante *threads*, *mutexes* y sincronización.

Ejemplo literal permitido: “Coders do not communicate with each other.”

II. AI Instructions

Se explica cómo usar herramientas de IA de forma responsable:

- Utilizar IA para tareas repetitivas.
- Desarrollar habilidades de prompting.
- Revisar y validar siempre lo generado.
- Buscar *peer review*.
- Entender los riesgos, sesgos y limitaciones de la IA.

Incluye ejemplos de *good practice* y *bad practice* sobre cómo apoyarse en IA sin perder comprensión real del código.

III. Common Instructions

Reglas generales del proyecto:

- Todo debe estar escrito en C y cumplir la *Norm*.
- No se permiten *segfaults*, *double free*, etc.
- No debe haber *memory leaks*.
- Se requiere un *Makefile* con reglas estándar: *NAME*, *all*, *clean*, *fclean*, *re*.
- Si se usa *libft*, debe incluirse correctamente.
- Se recomienda crear programas de test.
- Solo se evalúa lo que está en el repositorio Git.

IV. Overview

Resumen del funcionamiento de la simulación:

- Los coders están sentados en círculo.
- Cada coder alterna entre *compile* → *debug* → *refactor* → *compile*.
- Para *compile*, necesita dos dongles: el izquierdo y el derecho.
- Tras compilar, libera los dongles y pasa a *debug* y luego *refactor*.
- Si un coder tarda demasiado en volver a *compile*, sufre *burnout*.
- La simulación termina si alguien se quema o si todos alcanzan el número requerido de compilaciones.
- No hay comunicación entre coders.

V. Global Rules

El programa recibe los siguientes argumentos:

1. `number_of_coders`
2. `time_to_burnout`
3. `time_to_compile`
4. `time_to_debug`
5. `time_to_refactor`
6. `number_of_compiles_required`
7. `dongle_cooldown`
8. `scheduler (fifo o edf)`

Reglas clave:

- No se permiten variables globales.
- Los logs deben seguir un formato exacto.
- El mensaje de *burnout* debe imprimirse en menos de 10 ms desde que ocurre.
- Los coders deben evitar *burnout*.
- Se debe rechazar cualquier input inválido.

Ejemplo literal permitido: “timestamp_in_ms X has taken a dongle”

VI. Mandatory Part

Implementación obligatoria:

- Un *thread* por coder.

- Un dongle entre cada par de coders (dos por coder, excepto si solo hay uno).
- Cada dongle debe tener su propio `pthread_mutex_t`.
- Se puede usar `pthread_cond_t` para gestionar colas de espera.
- Debe implementarse *dongle cooldown*.
- Debe implementarse un *scheduler* justo:
 - `fifo`: orden de llegada.
 - `edf`: *Earliest Deadline First*.
- Debe garantizarse *liveness* bajo `edf`.
- Un *monitor thread* detecta *burnout* con precisión.
- Los logs deben estar serializados con un mutex.
- Se debe implementar una *priority queue* (heap) manual.
- No debe haber fugas de memoria.

VII. README Requirements

El README debe incluir:

- Línea inicial en cursiva indicando que es parte del currículo 42.
- Descripción del proyecto.
- Instrucciones de compilación y ejecución.
- Recursos y explicación de cómo se usó IA.
- Secciones adicionales obligatorias:
 - *Blocking cases handled*: deadlocks, starvation, cooldown, burnout detection, log serialization.
 - *Thread synchronization mechanisms*: explicación de mutexes, cond vars, eventos, y cómo se evita *race conditions*.

VIII. Submission and Peer Evaluation

- Se entrega en el repositorio Git.
- Durante la defensa pueden pedirte modificar una parte del código para demostrar comprensión.
- Las modificaciones deben ser rápidas y simples.